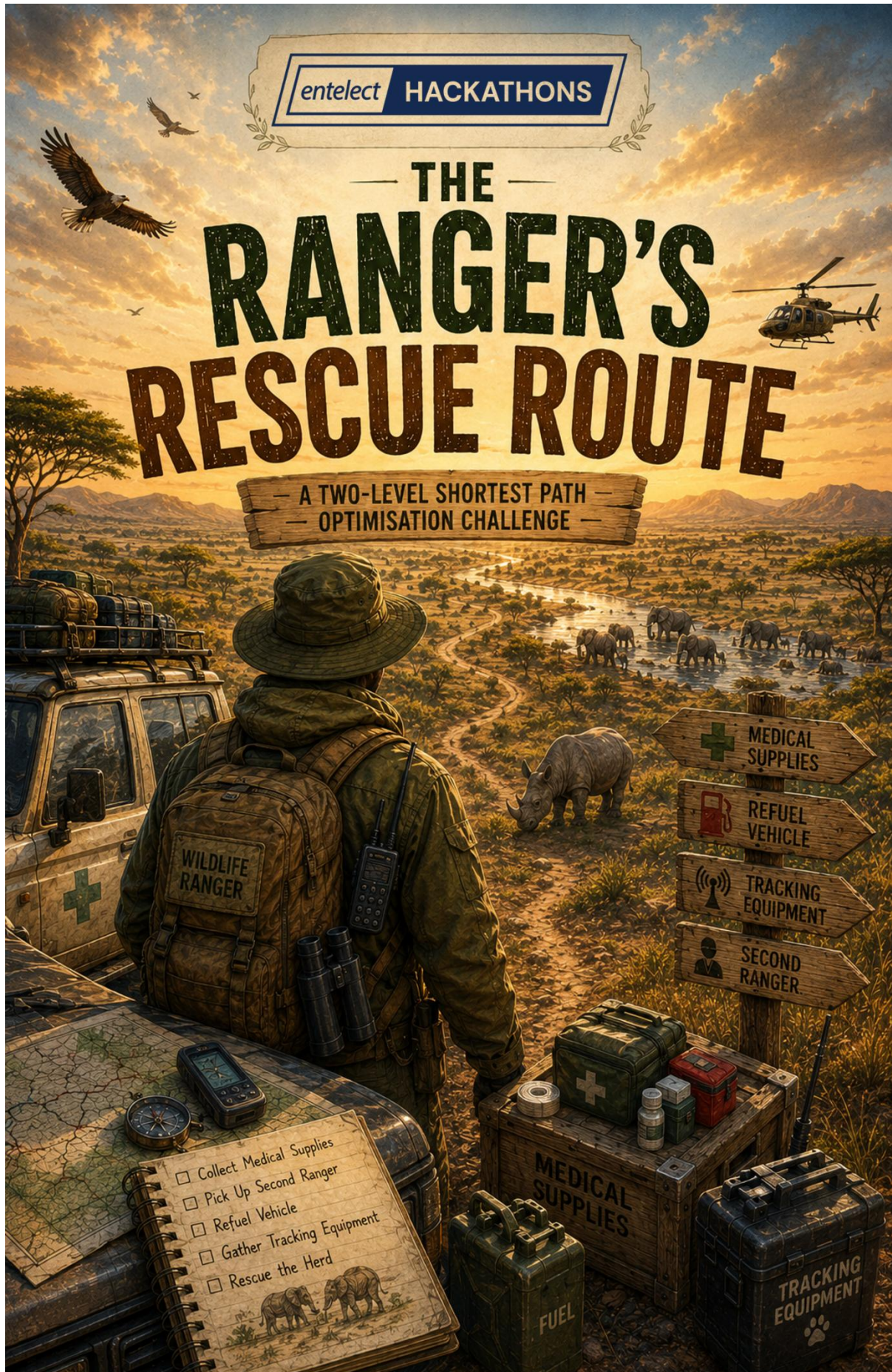


entelect HACKATHONS

THE RANGER'S RESCUE ROUTE

— A TWO-LEVEL SHORTEST PATH
— OPTIMISATION CHALLENGE —



The Ranger's Rescue Route

A Two-Level Shortest Path Optimisation Challenge

Introduction

You are a wildlife ranger responsible for navigating conservation areas using a network of trails. Each trail has a travel cost, and your job is to determine the most efficient routes using shortest path algorithms.

Your mission has two levels:

- **Level 1:** A small reserve — find the shortest path between two points.
- **Level 2:** A larger savannah — including risk-adjusted trail weights and a multi-stop required route.

Your objective is to apply shortest path principles to find efficient travel routes through the trail network.

Understanding the Input Graph

Level graphs are provided as an **adjacency list**.

For adjacency list input:

- Each key is a node name.
- Each value is a list of neighbour objects.
- Level 1 neighbours contain node and weight (travel time).
- Level 2 neighbours contain node, time, and risk (effective weight = time + risk).

The graph is undirected — every edge appears in both directions.

How to Submit

Your Answer File

Create a plain text file (e.g., answer.txt or submission.json) containing the **JSON** object described below. This is what is uploaded to the hackathon website to be scored.

Answer Format

Submit a **JSON** object with a single route array listing the node names in order from start to end:

```
{
  "route": ["A", "D", "E", "B"]
}
```

The solver will calculate the route cost from the graph — you do not need to include it.

Your Source Code

You can use **any programming language** you like. Along with your answer file, upload a **.zip** of your source code to the hackathon website.

In a real hackathon, this code is used to verify that your submitted answer was genuinely produced by your algorithm (not hand-crafted). For this **practice hackathon**, your code will **not** be reviewed — but please submit it anyway to get familiar with the format.

Required: Include a **README** in your zip that explains how to run your code so that a reviewer could easily reproduce your output.

Dijkstra's Algorithm — Getting Started

Dijkstra's algorithm finds the shortest path between two nodes in a weighted graph. It's the right tool for both levels of this challenge.

How It Works

1. Assign a cost of 0 to the start node and ∞ to every other node.
2. Always visit the unvisited node with the **lowest current cost** next.
3. For each neighbour of that node, calculate what it would cost to reach it via the current node. If that's cheaper than its current recorded cost, update it and remember how you got there.
4. Repeat until you reach the destination, then trace back through your recorded path.

Level 2 Note

For Level 2, use **time + risk** as the edge cost instead of just weight. Find the best ordering of the required stations, run Dijkstra for each consecutive leg, and concatenate the results into one continuous route.

Level 1 — The Small Reserve

Story Context

A distress call comes in from a small wildlife reserve. A young rhino is stuck near Point **B**, and you must reach it quickly from Point **A**.

Problem Setup

You are given a connected, weighted, undirected graph:

- Nodes represent locations.
- Edges represent trails.
 - Edge weight = travel time.
- All trails are safe (no risk penalties in Level 1).

Your Tasks

1. Find the shortest path from A to B through the trail network.
2. Submit the route — the solver calculates the cost and automatically scores you.

Learning Focus

- Shortest path algorithms (e.g. Dijkstra's).
- Weighted graph traversal.

Level 1 Graph

Nodes

A, B, C, D, E, F

Edge List (travel time only)

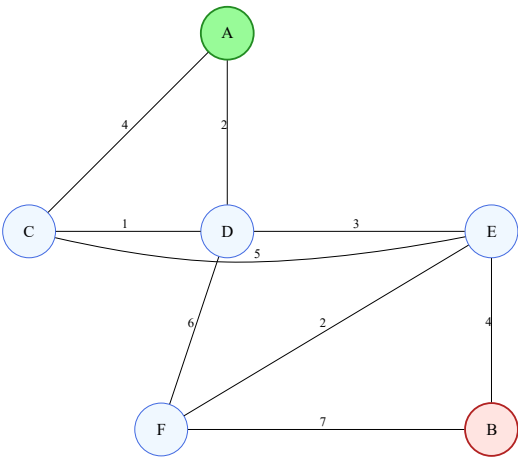
A	C	4
A	D	2
C	D	1
C	E	5
D	E	3
D	F	6
E	F	2
B	E	4
B	F	7

Adjacency List (same graph)

```
{
  "A": [{ "node": "C", "weight": 4 }, { "node": "D", "weight": 2 }],
  "B": [{ "node": "E", "weight": 4 }, { "node": "F", "weight": 7 }],
  "C": [{ "node": "A", "weight": 4 }, { "node": "D", "weight": 1 }, { "node": "E", "weight": 5 }],
  "D": [{ "node": "A", "weight": 2 }, { "node": "C", "weight": 1 }, { "node": "E", "weight": 3 }, { "node": "F", "weight": 6 }],
  "E": [{ "node": "C", "weight": 5 }, { "node": "D", "weight": 3 }, { "node": "F", "weight": 2 }, { "node": "B", "weight": 4 }],
  "F": [{ "node": "D", "weight": 6 }, { "node": "E", "weight": 2 }, { "node": "B", "weight": 7 } ]
}
```

Level 1 - Weighted Graph

Start: A



End: B

Level 2 — The Great Savannah: Elephant Migration Crisis

Story Context

You are deployed to the Great Savannah. A herd of elephants is moving toward Point **B**, and you must reach them from Point **A** as quickly as possible.

Along the way you must visit four stations, but the **order is up to you**:

- **S1** — collect medical supplies.
- **S2** — pick up a second ranger.
- **S3** — refuel your vehicle.
- **S4** — gather tracking equipment.

The challenge: visit all four stations and reach **B** in the **minimum total travel time**. The optimal order to visit them is not obvious — the graph topology determines which ordering is cheapest.

Problem Setup

You are given a connected, weighted, undirected graph.

Each edge has:

- Time (minutes)
- Risk (0–5)

Risk-adjusted edge cost:

effective weight = time + risk

Your Tasks

1. Find the **optimal ordering** of the four stations (S1, S2, S3, S4).
2. For each consecutive leg in your chosen ordering, find the shortest path using effective weights.
3. Concatenate the legs into one continuous route and submit it.

Hint: There are $4! = 24$ possible orderings. The alphabetical order $S1 \rightarrow S2 \rightarrow S3 \rightarrow S4$ is **not** optimal.

Learning Focus

- Dijkstra's algorithm with modified edge weights.
- Combinatorial optimisation over stop orderings.
- Understanding how graph topology makes some orderings much cheaper than others.

Level 2 Graph

Nodes

A, B, S1, S2, S3, S4, P1...P12

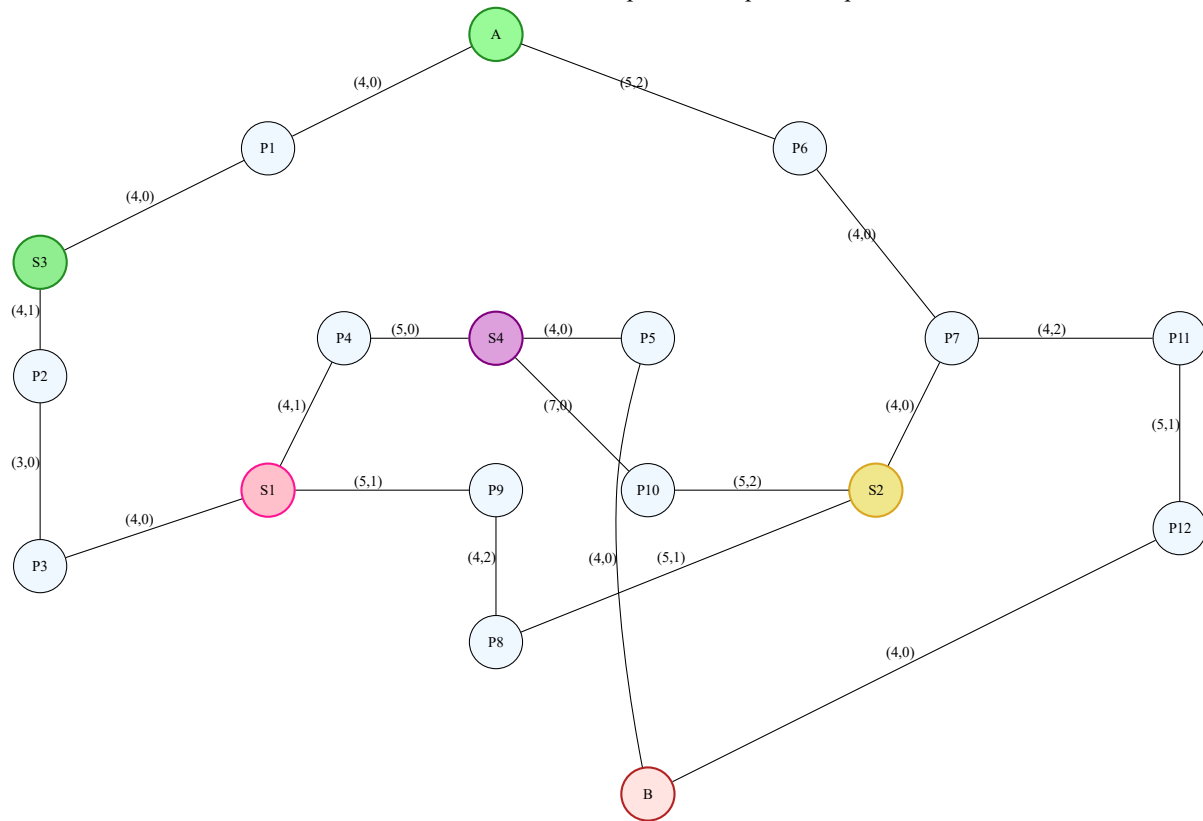
Edge List (time risk)

A	P1	4	0
A	P6	5	2
P1	S3	4	0
S3	P2	4	1
P2	P3	3	0
P3	S1	4	0
S1	P4	4	1
P4	S4	5	0
S4	P5	4	0
P5	B	4	0
P6	P7	4	0
P7	S2	4	0
P7	P11	4	2
S2	P8	5	1
P8	P9	4	2
P9	S1	5	1
S2	P10	5	2
P10	S4	7	0
P11	P12	5	1
P12	B	4	0

Adjacency List (same graph)

```
{
  "A": [{"node": "P1", "time": 4, "risk": 0}, {"node": "P6", "time": 5, "risk": 2}],
  "B": [{"node": "P5", "time": 4, "risk": 0}, {"node": "P12", "time": 4, "risk": 0}],
  "S1": [{"node": "P3", "time": 4, "risk": 0}, {"node": "P4", "time": 4, "risk": 1}, {"node": "P9", "time": 5, "risk": 1}],
  "S2": [{"node": "P7", "time": 4, "risk": 0}, {"node": "P8", "time": 5, "risk": 1}, {"node": "P10", "time": 5, "risk": 2}],
  "S3": [{"node": "P1", "time": 4, "risk": 0}, {"node": "P2", "time": 4, "risk": 1}],
  "S4": [{"node": "P4", "time": 5, "risk": 0}, {"node": "P5", "time": 4, "risk": 0}, {"node": "P10", "time": 7, "risk": 0}],
  "P1": [{"node": "A", "time": 4, "risk": 0}, {"node": "S3", "time": 4, "risk": 0}],
  "P2": [{"node": "S3", "time": 4, "risk": 1}, {"node": "P3", "time": 3, "risk": 0}],
  "P3": [{"node": "P2", "time": 3, "risk": 0}, {"node": "S1", "time": 4, "risk": 0}],
  "P4": [{"node": "S1", "time": 4, "risk": 1}, {"node": "S4", "time": 5, "risk": 0}],
  "P5": [{"node": "S4", "time": 4, "risk": 0}, {"node": "B", "time": 4, "risk": 0}],
  "P6": [{"node": "A", "time": 5, "risk": 2}, {"node": "P7", "time": 4, "risk": 0}],
  "P7": [{"node": "P6", "time": 4, "risk": 0}, {"node": "S2", "time": 4, "risk": 0}, {"node": "P11", "time": 4, "risk": 2}],
  "P8": [{"node": "S2", "time": 5, "risk": 1}, {"node": "P9", "time": 4, "risk": 2}],
  "P9": [{"node": "P8", "time": 4, "risk": 2}, {"node": "S1", "time": 5, "risk": 1}],
  "P10": [{"node": "S2", "time": 5, "risk": 2}, {"node": "S4", "time": 7, "risk": 0}],
  "P11": [{"node": "P7", "time": 4, "risk": 2}, {"node": "P12", "time": 5, "risk": 1}],
  "P12": [{"node": "P11", "time": 5, "risk": 1}, {"node": "B", "time": 4, "risk": 0}]
}
```

Level 2 - Time & Risk Graph with Required Stops



Scoring

Each level is scored out of 100 points, giving a maximum total of 200 points.

How it works

Your score is based on how close your submitted route cost is to the known optimal route cost.

$$\text{score} = (\text{optimal_route_cost} / \text{your_route_cost}) * 100$$

- If your route cost matches the optimal, you score **100**.
- The further your route cost is from the optimal, the lower your score.
- The optimal is the true shortest path, so your cost cannot be lower — scores above 100 are not possible.

Level 1 Score (out of 100)

$$\text{level_1_score} = (\text{optimal_route_cost_L1} / \text{your_route_cost_L1}) * 100$$

Where $\text{optimal_route_cost_L1} = 9$.

Level 2 Score (out of 100)

$$\text{level_2_score} = (\text{optimal_route_cost_L2} / \text{your_route_cost_L2}) * 100$$

Where $\text{optimal_route_cost_L2} = 60$.

Total Score

$$\text{total_score} = \text{level_1_score} + \text{level_2_score}$$

Maximum total: **200 points**.